

ME315A_FarahLab05

Mateus Farah

Lab 05

```
library(tidyverse)
```

Warning: pacote 'tidyverse' foi compilado no R versão 4.6.1

Warning: pacote 'ggplot2' foi compilado no R versão 4.6.1

Warning: pacote 'forcats' foi compilado no R versão 4.6.1

```
-- Attaching core tidyverse packages ----- tidyverse 2.0.0 --  
v dplyr      1.2.1      v readr      2.2.0  
v forcats    1.0.1      v stringr    1.6.0  
v ggplot2    4.0.3      v tibble     3.3.1  
v lubridate  1.9.5      v tidyr      1.3.2  
v purrr      1.2.2
```

```
-- Conflicts ----- tidyverse_conflicts() --
```

```
x dplyr::filter() masks stats::filter()
```

```
x dplyr::lag()     masks stats::lag()
```

```
i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become
```

```
library(leaflet)
```

Warning: pacote 'leaflet' foi compilado no R versão 4.6.1

```
library(lubridate)  
library(readr)  
library(scales)
```

Anexando pacote: 'scales'

O seguinte objeto é mascarado por 'package:purrr':

discard

O seguinte objeto é mascarado por 'package:readr':

col_factor

```
analisa_aeronave <- function(tail_number, arquivo, arquivo_airports = NULL) {  
  
  # 1. Ajusta o caminho do arquivo de aeroportos caso não tenha sido informado  
  if (is.null(arquivo_airports)) {  
    arquivo_airports <- file.path(dirname(arquivo), "airports.csv")  
  }  
  
  # Validação prévia da existência dos arquivos no disco  
  if (!file.exists(arquivo)) {  
    stop("Arquivo de voos não encontrado no caminho: ", arquivo)  
  }  
  if (!file.exists(arquivo_airports)) {  
    stop("Arquivo de aeroportos não encontrado no caminho: ", arquivo_airports)  
  }  
  
  # 2. Leitura por blocos com tipos de colunas explicitados para evitar inconsistências  
  callback_filtragem <- function(dados_chunk, pos) {  
    dados_chunk %>% filter(TAIL_NUMBER == tail_number)  
  }  
  # Lê o CSV em pedaços de 100k linhas pra não travar a memória  
  voos_aeronave <- read_csv_chunked(  
    file = arquivo,  
    callback = DataFrameCallback$new(callback_filtragem),  
    chunk_size = 100000,  
    col_types = cols(  
      YEAR = col_integer(),  
      MONTH = col_integer(),  
      DAY = col_integer(),  
      TAIL_NUMBER = col_character(),  
      ORIGIN_AIRPORT = col_character(),  
      DESTINATION_AIRPORT = col_character(),  
    )  
  )  
}
```

```

    DEPARTURE_TIME = col_character(),
    AIR_TIME = col_double(),
    DISTANCE = col_double(),
    AIRLINE = col_character(),
    FLIGHT_NUMBER = col_character()
  ),
  show_col_types = FALSE
)
# Trava se não achar nenhum voo desse avião
if (nrow(voos_aeronave) == 0) {
  stop("Nenhum registro encontrado para o código de aeronave: ", tail_number)
}

# 3. Leitura das coordenadas dos aeroportos
dados_aeroportos <- read_csv(
  arquivo_airports,
  col_types = cols(
    IATA_CODE = col_character(),
    AIRPORT = col_character(),
    LATITUDE = col_double(),
    LONGITUDE = col_double()
  ),
  show_col_types = FALSE
) %>%
  select(IATA_CODE, AIRPORT, LATITUDE, LONGITUDE) %>%
  drop_na(LATITUDE, LONGITUDE)

# 4. Tratamento dos dados no formato Tidy
tabela_voos_tidy <- voos_aeronave %>%
  filter(!is.na(DEPARTURE_TIME), !is.na(AIR_TIME), AIR_TIME > 0) %>%
  mutate(
    # Trata horários como 2400 (converter para 0000) evitando falha no make_datetime
    hora_num = as.numeric(DEPARTURE_TIME),
    hora_num = if_else(hora_num == 2400, 0, hora_num),
    hora_str = sprintf("%04d", hora_num),
    hora = as.numeric(substr(hora_str, 1, 2)),
    minuto = as.numeric(substr(hora_str, 3, 4)),

    # Geração da data e hora
    data_hora_partida = make_datetime(YEAR, MONTH, DAY, hora, minuto),

    # Cálculo da velocidade média (mph)

```

```

    velocidade_media_mph = (DISTANCE / (AIR_TIME / 60))
  ) %>%
  filter(!is.na(data_hora_partida)) %>%
  arrange(data_hora_partida) %>%
  # Junção das coordenadas de origem e destino
  left_join(dados_aeroportos, by = c("ORIGIN_AIRPORT" = "IATA_CODE")) %>%
  rename(lat_origem = LATITUDE, lon_origem = LONGITUDE, nome_origem = AIRPORT) %>%
  left_join(dados_aeroportos, by = c("DESTINATION_AIRPORT" = "IATA_CODE")) %>%
  rename(lat_destino = LATITUDE, lon_destino = LONGITUDE, nome_destino = AIRPORT) %>%
  filter(!is.na(lat_origem), !is.na(lat_destino))

if (nrow(tabela_voos_tidy) == 0) {
  stop("Os voos encontrados não possuem coordenadas correspondentes no arquivo de aeroportos")
}

# 5. Criação do mapa interativo
aeroportos_visitados <- bind_rows(
  tabela_voos_tidy %>% select(codigo = ORIGIN_AIRPORT, nome = nome_origem, lat = lat_origem),
  tabela_voos_tidy %>% select(codigo = DESTINATION_AIRPORT, nome = nome_destino, lat = lat_destino)
) %>%
  distinct(codigo, .keep_all = TRUE)

mapa_trajeto <- leaflet() %>%
  addProviderTiles(providers$CartoDB.Positron) %>%
  addCircleMarkers(
    data = aeroportos_visitados,
    lng = ~lon, lat = ~lat,
    popup = ~paste0("<b>Aeroporto:</b> ", nome, " (", codigo, ")"),
    radius = 5, color = "#2c3e50", fillOpacity = 0.8
  )

# Desenha cada voo na ordem cronológica (linhas mais grossas = voos mais rápidos)
for (i in seq_len(nrow(tabela_voos_tidy))) {
  voo <- tabela_voos_tidy[i, ]

  espessura <- rescale(voo$velocidade_media_mph, to = c(1.5, 6), from = c(100, 600))

  popup_voo <- paste0(
    "<b>Voo nº:</b> ", i, " do ano<br>",
    "<b>Companhia:</b> ", voo$AIRLINE, " ", voo$FLIGHT_NUMBER, "<br>",
    "<b>Data/Hora:</b> ", format(voo$data_hora_partida, "%d/%m/%Y %H:%M"), "<br>",
    "<b>Trecho:</b> ", voo$ORIGIN_AIRPORT, " → ", voo$DESTINATION_AIRPORT, "<br>",
    "<b>Velocidade Média:</b> ", round(voo$velocidade_media_mph, 1), " mph"
  )
}

```

```

)

mapa_trajeto <- mapa_trajeto %>%
  addPolylines(
    lng = c(voo$lon_origem, voo$lon_destino),
    lat = c(voo$lat_origem, voo$lat_destino),
    color = "#3498db", weight = espessura, opacity = 0.6,
    popup = popup_voo
  )
}
# Entrega o resultado como lista nomeada igual pediu a especificação
return(list(
  tabela = tabela_voos_tidy,
  mapa = mapa_trajeto
))
}

```

```

resultado <- analisa_aeronave(
  tail_number = "N431WN",
  arquivo = "C:/Users/mateu/Downloads/flights.csv.zip",
  arquivo_airports = "C:/Users/mateu/Downloads/airports.csv"
)

head(resultado$tabela)

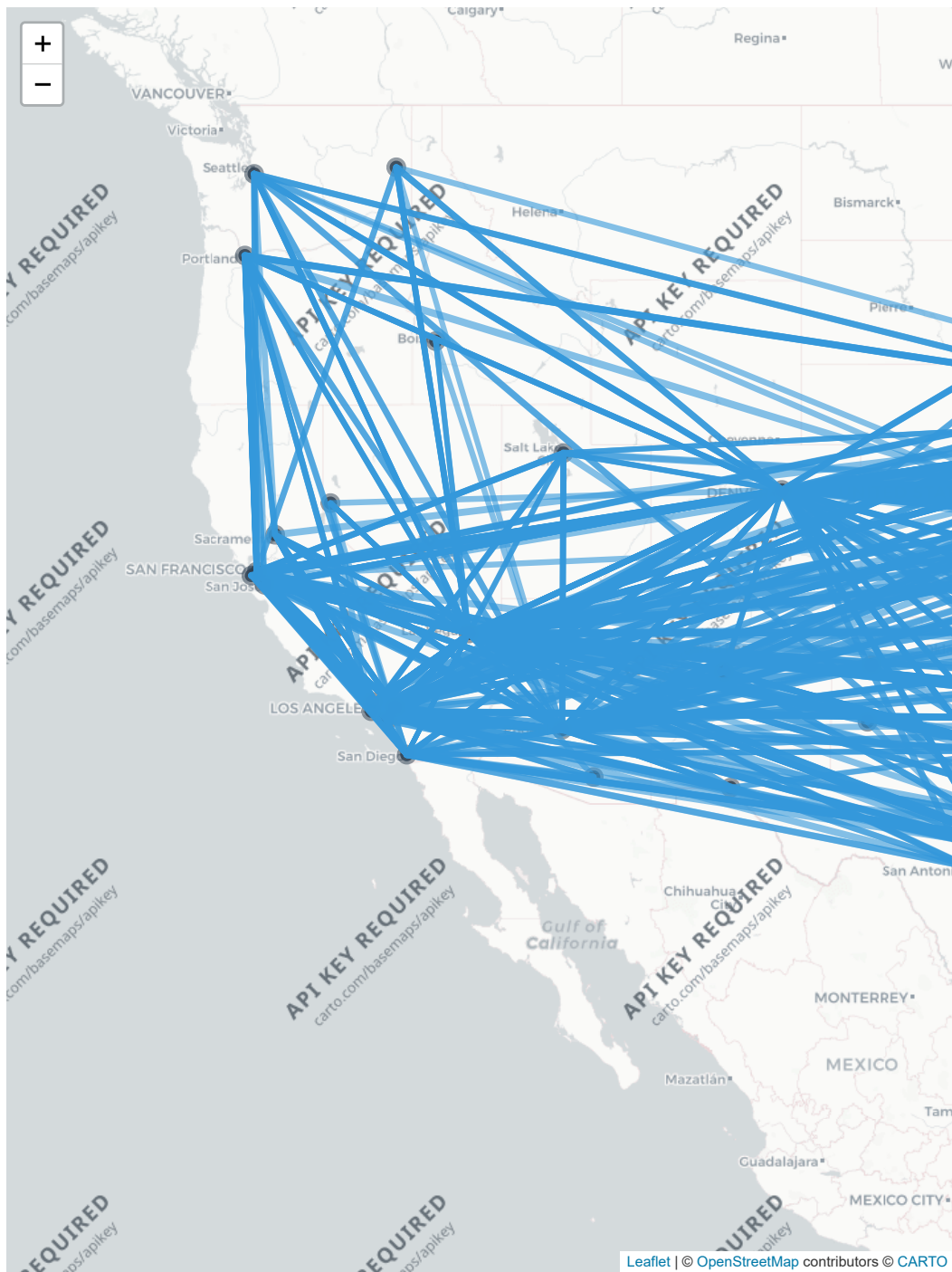
```

```

# A tibble: 6 x 43
  YEAR MONTH DAY DAY_OF_WEEK AIRLINE FLIGHT_NUMBER TAIL_NUMBER ORIGIN_AIRPORT
  <int> <int> <int>      <dbl> <chr>      <chr>      <chr>      <chr>
1  2015     1     1          4 WN         1762      N431WN      DCA
2  2015     1     1          4 WN         3564      N431WN      HOU
3  2015     1     1          4 WN         3564      N431WN      BOS
4  2015     1     1          4 WN         3564      N431WN      BWI
5  2015     1     2          5 WN         1793      N431WN      MCI
6  2015     1     2          5 WN         1793      N431WN      LAX
# i 35 more variables: DESTINATION_AIRPORT <chr>, SCHEDULED_DEPARTURE <chr>,
# DEPARTURE_TIME <chr>, DEPARTURE_DELAY <dbl>, TAXI_OUT <dbl>,
# WHEELS_OFF <chr>, SCHEDULED_TIME <dbl>, ELAPSED_TIME <dbl>, AIR_TIME <dbl>,
# DISTANCE <dbl>, WHEELS_ON <chr>, TAXI_IN <dbl>, SCHEDULED_ARRIVAL <chr>,
# ARRIVAL_TIME <chr>, ARRIVAL_DELAY <dbl>, DIVERTED <dbl>, CANCELLED <dbl>,
# CANCELLATION_REASON <chr>, AIR_SYSTEM_DELAY <dbl>, SECURITY_DELAY <dbl>,
# AIRLINE_DELAY <dbl>, LATE_AIRCRAFT_DELAY <dbl>, WEATHER_DELAY <dbl>, ...

```

```
resultado$mapa
```



```
Sys.setenv(TZ = "America/Sao_Paulo")  
Sys.time()
```

```
[1] "2026-09-16 12:33:49 -03"
```